

# 计算机程序设计基础(2)

## --- C++程序设计(10)

孙甲松

[sunjiasong@tsinghua.edu.cn](mailto:sunjiasong@tsinghua.edu.cn)

电子工程系 信息认知与智能系统研究所  
罗姆楼6-104

电话: 62796193/13901216180

2020. 4.

1

## 8. 流类库与输入/输出

- ◆ I/O流的概念
- ◆ 一个程序必须有输入/输出，也就是与外界交换数据，这可以看成是数据在两个对象之间流动。
- ◆ 流(stream)是一种抽象，负责在两个对象之间建立联系，并管理数据的流动。
- ◆ 程序建立一个流对象，将流对象与文件对象建立连接(打开文件)，程序对流对象进行操作，而流对象通过文件系统对所连接的文件进行读写。
- ◆ 读操作在流数据抽象中被称为从流中提取。
- ◆ 写操作在流数据抽象中被称为向流中插入。

2

## 8.1 输出流

- 最重要的输出流有三个：
    - ostream 通用输出流类（标准输出）
    - ofstream 输出文件流类（向文件输出）
    - ostringstream 输出字符串流类（向字符串输出）
  - ostream类对象完成向标准设备的输出，包括：
    - cout 标准输出
    - cerr 标准错误输出，无缓冲，立即输出
    - clog 标准错误输出，有缓冲，缓冲区满时再输出
  - 插入运算符 << 对所有数据类型都已经预先设置好输出。
- 例如：

3

## 8.1 输出流

```
#include <iostream>
using namespace std;
void main( )
{ cerr << "Error!\n";
  cout << "OK!\n";
  clog << "clog!\n";
  cerr << "Error!\n";
  cout << "OK!\n";
}
```

VC6编译运行结果：

(此结果可能因为不同的编译系统有所不同！)

```
Error!
OK!
Error!
OK!
clog!
```

4

```
#include <iostream>
using namespace std;
void main( )
{ cerr << "Error!\n";
  cout << "OK!\n";
  clog << "clog!\n";
  cerr << "Error!\n";
  cout << "OK!\n";
}
```

VS2008和VS2012编译运行结果：

Error!

OK!

clog!

Error!

OK!

(看起来clog也无缓冲，立即输出了)

5

## 8.1 输出流

- ofstream类支持对文件的输出,步骤是：
  1. 构造一个ofstream类的对象 (文件引用为 fstream)
  2. 打开文件
  3. 在打开文件的同时指定输出方式：  
二进制方式/文本方式
- 打开文件的方式有4种：
  1. 可以在构造对象时利用其构造函数同时打开文件：  
ofstream of("filename", iosmode);
  2. 先构造静态对象，再使用其open成员函数打开文件：  
ofstream of; // 静态输出文件流对象  
of.open("filename", iosmode);
  3. 先构造动态对象，再使用其open成员函数打开文件：  
ofstream \*ofp=new ofstream; // 动态输出文件流对象  
ofp->open("filename", iosmode);
  4. 构造动态对象的同时，利用其构造函数打开文件：  
ofstream \*ofp=new ofstream("filename", iosmode);

6

## 8.1 输出流

用: `of.close()`; 或: `ofp->close()`;

关闭打开的文件, 便于用`of`、`*ofp`对象再打开别的文件。

### • 常用输出流成员函数有:

1. `open` 函数 ( `ios_base::` 是ISO C++方式, `ios::` 是普通C++方式)

打开文件, `of.open("filename", iosmode);`

普通方式 ( `#include <fstream.h>` ):

|                                |                    |
|--------------------------------|--------------------|
| <code>iosmode: ios::app</code> | 文件尾添加              |
| <code>ios::ate</code>          | 打开现存文件并移到文件尾       |
| <code>ios::in</code>           | 打开一个输入文件(保护文件)     |
| <code>ios::out</code>          | 写方式打开, 隐含模式, 可以省略  |
| <code>ios::trunc</code>        | 打开文件, 抹去原有内容       |
| <code>ios::binary</code>       | 二进制方式打开, 默认是文本方式打开 |
| <code>ios::nocreate</code>     | 打开已有文件, 否则失败       |
| <code>ios::noreplace</code>    | 文件不存在则生成, 否则失败     |

以上各方式可以用 | 运算符组合起来使用。

7

标准ISO C++命名空间方式:

```
#include <fstream>
```

```
using namespace std;
```

`iosmode:`

|                                  |                    |
|----------------------------------|--------------------|
| <code>ios_base::app</code>       | 文件尾添加              |
| <code>ios_base::ate</code>       | 打开现存文件并移到文件尾       |
| <code>ios_base::in</code>        | 打开一个输入文件(保护文件)     |
| <code>ios_base::out</code>       | 写方式打开, 隐含模式, 可以省略  |
| <code>ios_base::trunc</code>     | 打开文件, 抹去原有内容       |
| <code>ios_base::binary</code>    | 二进制方式打开, 默认是文本方式打开 |
| <code>ios_base::nocreate</code>  | 打开已有文件, 否则失败       |
| <code>ios_base::noreplace</code> | 文件不存在则生成, 否则失败     |

以上各方式可以通过 | 运算符组合起来使用。

注意: 如果写成: `of.open("filename")`

缺省打开方式, 则默认是以**文本写方式**打开文件。

8

## 8.1 输出流

### 2. `close` 函数

关闭打开的文件, 文件使用完毕必须关闭。

但`ofstream`类的析构函数也能自动完成文件关闭。

### 3. `put` 函数

输出一个字符到输出流。

`cout.put('A');` // 等价于 `cout << 'A';`

`of.put('A');` // 等价于 `of << 'A';`

### 4. `write` 函数

二进制方式写到文件输出流中。长度参数指定写出的字节数。系统将不管输出内容, 按照你指定的内存和长度写出到文件输出流中。

9

二进制方式写文件:

```
#include <fstream>
```

```
using namespace std;
```

```
struct Date
```

```
{ int m, d, y;
```

```
};
```

```
void main( )
```

```
{ Date dt = {4,21,2020}; // VS2008与VS2012写 ios::binary 也可以  
ofstream tfile("data.dat", ios_base::binary);
```

```
tfile.write((char *)&dt, sizeof dt);
```

```
// 注意: 一定要把输出对象的指针转成char *型按字节输出  
tfile.close( );
```

```
}
```

```
// 所生成的data.dat文件长度为12个字节, 3个int
```

10

## 8.1 输出流

利用`<<` 进行文本方式写文件:

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
struct Date
```

```
{ int m,d,y;
```

```
};
```

```
void main( )
```

```
{ Date dt = {4, 21, 2020};
```

```
ofstream tfile("data2.dat");
```

```
//不指定打开方式, 则默认是文本写方式
```

```
tfile << dt.m <<','<< dt.d <<','<< dt.y <<endl; //输出到文件
```

```
cout << dt.m <<','<< dt.d <<','<< dt.y <<endl; //输出到屏幕
```

```
tfile.close( );
```

```
}
```

```
// 所生成的data2.dat文件内容将是 4,21,2020
```

11

## 8.1 输出流

### 5. `seekp`和`tellp`函数

`tellp()`返回当前文件指针位置, 便于建立索引文件, 以便快速定位、读写。(从文件头开始的相对位置, 从0开始以字节计数)

`seekp(n, pos)`设置指针位置, 便于快速定位随机读写

`pos`取值: `ios_base::beg` 从文件头开始

`ios_base::end` 从文件尾开始

`ios_base::cur` 从文件指针的当前位置开始

若写为: `seekp(n)`

缺省值默认是 `ios_base::beg` 从文件头开始

### 6. `flush` 函数

与C语言的`fflush`函数类似, 清空I/O缓冲区到文件中, 防止出现同时写/读同一文件时, 内容不同步。

12

## 8.1 输出流

### 7. clear函数

清除（复位）所有错误标记位，包括：goodbit, failbit, eofbit, badbit，此函数很重要，许多时候不能忽视。尤其是在发生输入输出错误后，需要利用clear函数让输入输出流对象恢复正常。

**注意：**同一个流对象在打开过一个文件并关闭后，再打开另一个文件前，最好先调用一次clear函数，给流对象复位。

### 8. 错误处理函数

eof() 遇到文件尾则返回一个非0值。

bad(), fail(), good() 等自己看一下。

其它输出流的成员函数这里不再多讲。

13

在输入输出流对象名tfile并输入'.'后，系统会提示：



其中，前面有 为成员函数。

14

### 文本输出格式的控制符

#### 1. setw操纵符和width成员函数

```
#include <iostream>
#include <iomanip>
using namespace std;
void main()
{ double values[] = {1.23, 35.36, 653.7, 4358.24};
  for(int i=0;i<4;i++)
  { cout << setw(10) << values[i] << endl;
    // setw(10) 设置随后的输出项宽度为10个字符
  }
}
```

运行结果为：

```
1.23
35.36
653.7
4358.24
```

15

### 文本输出格式的控制符

```
#include <iostream>
using namespace std;
void main()
{ double values[] = {1.23, 35.36, 653.7, 4358.24};
  for(int i=0;i<4;i++)
  { cout.width(10); //width设置随后的输出项宽度为10个字符
    cout << values[i] << endl;
  }
}
```

运行结果为：

```
1.23
35.36
653.7
4358.24
```

16

## 8.1 输出流

```
#include <iostream>
using namespace std;
void main()
{ double values[] = {1.23, 35.36, 653.7, 4358.24};
  for(int i=0;i<4;i++)
  { cout.width(10);
    cout.fill('*'); //用*填充空白之处
    cout << values[i] << endl;
  }
}
```

运行结果为：

```
*****1.23
*****35.36
*****653.7
****4358.24
```

17

## 8.1 输出流

### 2. 对齐方式: (需要 #include <iomanip> )

setiosflags(ios\_base::left) 左对齐

setiosflags(ios\_base::right) (默认方式)

### 3. 精度

setprecision(i) 设置随后输出的浮点数i位小数

setiosflags(ios\_base::fixed) 小数方式

setiosflags(ios\_base::scientific) 科学方式

### 4. 进制

dec、oct、hex

分别以十进制、八进制、十六进制方式输出，默认为十进制

setiosflags(ios\_base::uppercase)

将字母按大写方式输出十六进制数

18

## 8.1 输出流

### 5. 关于setiosflags的使用

- ◆ setiosflags不同于width和setw，它的影响是持久的或者说只要进行了某种设置，除非取消将一直起作用。
- ◆ 用resetiosflags操作符关闭前面setiosflags设置的标志，重新恢复默认值时，setiosflags设置的标志影响力才终止。
- ◆ 此时下一个setiosflags设置才会起作用。例如程序：

```
#include<iostream>
#include<iomanip>
using namespace std;
int main( )
{ double a=123.4567890;
  cout<<setiosflags(ios_base::fixed)<<a<<<endl;
  cout<<setiosflags(ios_base::scientific)<<setprecision(5)
    <<a<<<endl;
  return 0;
}
```

19

运行结果是：

```
123.456789
0x1.edd3cp+6
请按任意键继续...
```

第二个数的输出结果看起来非常令人匪夷所思，改为：

```
#include<iostream>
#include<iomanip>
using namespace std;
int main( )
{ double a=123.4567890;
  cout<<setiosflags(ios_base::fixed)<<a<<<endl;
  cout<<resetiosflags(ios_base::fixed); //关闭setiosflags设置的标志
  cout<<setiosflags(ios_base::scientific)<<setprecision(5)
    <<a<<<endl;
  return 0;
}
```

才使得后一个语句输出正常结果：

20

如果在关闭setiosflags设置的ios\_base::fixed后，不设置setiosflags(ios\_base::scientific)，程序改为：

```
#include<iostream>
#include<iomanip>
using namespace std;
int main( )
{ double a=123.4567890, b=1234567.890;;
  cout<<setiosflags(ios_base::fixed)<<a<<' '<<b<<<endl;
  cout<<resetiosflags(ios_base::fixed); //关闭setiosflags设置的标志
  cout<<a<<' '<<b<<<endl;
}
```

运行结果是：

```
123.456789 1234567.890000
123.457 1.23457e+006
```

请按任意键继续...

如果不指定ios\_base::fixed或ios\_base::scientific格式，相当于C语言printf输出时不指定%f或%e输出格式，则自动用%g输出格式，始终按6位有效数字输出浮点数，直至按科学方式输出浮点数。

21

## 8.2 输入流

- 最重要的输入流有三个：

- istream 通用输入流类（标准输入）
- ifstream 输入文件流类（从文件输入）
- istreamstringstream 输入字符串流类（从字符串输入）

- istream类对象完成从标准设备的输入：

1. cin是其派生类istream\_withassign的对象。
2. 提取运算符>>对所有数据类型都已经预先设置好。
3. 还可以使用dec, oct, hex改变输入流的输入方式。
4. get、getline的功能是从输入流一次读入多个字符，并包括控制符，可以从标准设备/文件输入。

22

## 8.2 输入流

- ifstream类支持对文件的输入，步骤是：

1. 构造一个ifstream类的对象（文件头部引用<fstream>）
2. 打开文件
3. 在打开文件之前或之后指定输入方式：  
二进制方式/文本方式

- 打开文件的方式有4种：

1. 可以在构造对象时利用其构造函数同时打开文件：  
ifstream inf("filename", iosmode);
2. 先构造静态对象，再使用其open成员函数打开文件：  
ifstream inf; //静态输入文件流对象  
inf.open("filename", iosmode);
3. 先构造动态对象，再使用其open成员函数打开文件：  
ifstream \*infp=new ifstream; //动态输入文件流对象  
infp->open("filename", iosmode);
4. 在构造动态对象时利用其构造函数同时打开文件：  
ifstream \*infp=new ifstream("filename", iosmode);

23

## 8.2 输入流

文件打开方式iosmode:

普通方式（非国际标准方式）：

ios::in（默认方式）

ios::binary 二进制方式（默认文本方式）

ISO C++标准命名空间方式：

ios\_base::in（默认方式）

ios\_base::binary 二进制方式（默认文本方式）

打开的每一个文件，对应用close关闭。

几个常用函数：

### 1. read函数

read函数实现二进制读。按字节数来读入指定区域，数据格式不转换。

24

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
void main( )
{ struct
{ double salary;
char name[23];
} employee1, employee2;
employee1.salary=8000;
strcpy(employee1.name, "San Zhang");
ofstream outfile("payroll.dat", ios_base::binary);
outfile.write((char *)&employee1, sizeof(employee1));
outfile.close( );
// outfile.clear( );
```

25

```
ifstream is("payroll.dat", ios_base::binary);
if (is.good( )) // if (is.is_open( ))
{ is.read((char *)&employee2, sizeof(employee2));
cout << employee2.name << ' '
<< employee2.salary << endl;
}
else
cout << "ERROR: Can't open file: payroll.dat" << endl;
is.close( );
}
```

运行结果:  
San Zhang 8000  
请按任意键继续. . .  
payroll.dat文件长度为32个字节

26

## 8.2 输入流

### 2. seekg和tellg函数

tellg( )返回当前文件指针位置 (从文件头开始的  
相对位置, 从0开始以字节计数)

seekg(n, pos)设置指针位置, 便于随机读写

pos取值: ios\_base::beg 文件头开始

ios\_base::end 文件尾开始

ios\_base::cur 文件中的当前位置开始

写为: seekg(n) 缺省值默认是ios\_base::beg从文件头开始  
这里 n 是字节数。

如果要跳过1个double数, n应该是 sizeof(double)

如果要跳过1个int数, n应该是 sizeof(int)

如果要跳过5个int数, n应该是 sizeof(int)\*5

27

```
#include <iostream>
#include <fstream>
using namespace std;
void main( )
{ char ch;
ifstream tfile("payroll.dat", ios_base::binary);
if(tfile.good( ))
{ tfile.seekg(sizeof(double), ios_base::beg); // tfile.seekg(8);
while (tfile.good( )) // 注意: 这里不能当正常出口
{ // 遇到文件结束或读取操作失败时结束读操作
tfile.get(ch); // 或者 ch = tfile.get( );
if (!ch) break; // 如果读到'\0' 字符串结束符则退出循环
cout << ch;
}
}
else cout << "ERROR: Cannot open file: payroll.dat" << endl;
tfile.close( );
}
```

运行结果:  
San Zhang请按任意键继续. . .

28

## 8.2 输入流

### 3. ignore函数

ignore(n, 终止字符) 跳过输入流中的n个字符, 或直到指定的终止字符 (终止字符也跳过), 终止字符可以是任意字符, 但通常是回车符'\n'。

- 大家上机经常遇到的一个问题是, 如果想用cin>>n;从键盘上读入一个数, 不小心输入了非数字字符, 此时如何清空输入缓冲区, 以便于能重新用cin>>n;正确读入数呢? 方法如下:

cin.clear( ); //清除std::cin的错误状态

cin.ignore(numeric\_limits<streamsize>::max( ), '\n');

需要引用头文件:

#include <iostream>

#include <limits>

using namespace std;

29

```
#include <iostream>
#include <limits>
using namespace std;
int main( )
{ int a;
while(cout<<"input a integer (1-10) :", cin>>a,
!(a>=1 && a<=10) || cin.fail( ))
{
cout<<"try again!"<<endl;
cin.clear( ); // 清除std::cin的错误状态,
// 如果不加此语句后果是很严重的.....
cin.ignore(numeric_limits<streamsize>::max( ), '\n');
}
return 0;
}
```

30

## 8.2 输入流

### 4. sync函数

sync() 清空输入输出缓冲区，把输入丢掉，把输出打印出来（或送到文件中）。

- 还是上面大家经常遇到的问题，如果想用  
cin >> n;  
从键盘上读入一个数，不小心输入了非数字字符，此时如何清空输入缓冲区，以便于能重新用cin>>n;正确读入数呢？方法如下：

```
cin.clear( ); //清除std::cin的错误状态
cin.sync( ); //清空输入缓冲区
这两个函数要一起使用，缺一不可！
```

31

```
#include <iostream>
using namespace std;
void main( )
{ int a;
  while(cout<<"input a integer (1-10) :", cin>>a,
        !(a>=1 && a<=10) || cin.fail( ))
  {
    cout<<"try again!"<<endl;
    cin.clear( ); //清除std::cin对象的错误状态
    // 如果不加此语句后果是很严重的.....
    cin.sync( ); //清空输入缓冲区
  }
}
```

其中cin.sync( );这一句必不可少，因为所有从标准输入设备输入的数据都是先保存在缓冲区中，然后istream对象再从缓冲区中进行提取。如果不清空缓存，下次在读取数据的时候又会再次产生错误，也会陷入死循环。

32

## 8.2 输入流

### 5. fail函数

fail() 是当cin尝试输入某个数据失败后，会使得输入流产生一个错误状态，从而使cin.fail()为真，否则为假。

fail函数是一个bool型的状态函数。可由cin.fail()判断上一次的输入是否成功。

其余good, bad之类的状态函数不再讲，它们通过检测goodbit, badbit位得到结果。fail和eof函数是通过检测failbit, eofbit位得到结果。这些函数自己看一下。

33

```
#include <iostream>
#include <fstream>
using namespace std;
void main( )
{ char ch;
  ifstream tfile("payroll.dat", ios_base::binary);
  if(tfile)
  { tfile.seekg(sizeof(double), ios_base::beg); // 从文件头跳8个字节
    while (1)
    { // 读取操作失败时结束读操作
      tfile.get(ch); // 或写为: ch = tfile.get( ); 读入一个字符
      if (tfile.fail()) break; // 如果读入失败则退出循环
      cout << ch;
    }
  }
  else cout << "ERROR: Cannot open file: payroll.dat" << endl;
  tfile.close( );
}
```

运行结果：

San Zhang 烫烫烫烫烫烫烫请按任意键继续...

34

```
#include <iostream>
#include <fstream>
using namespace std;
void main( )
{ char ch[50];
  ifstream tfile("payroll.dat", ios_base::binary);
  if(tfile)
  { tfile.seekg(sizeof(double), ios_base::beg); // 从文件头跳8个字节
    while (1)
    { // 读取操作失败时结束读操作
      tfile.get(ch, 50, '\0'); // 一次全部读入
      if (tfile.fail()) break; // 如果读入失败则退出循环
      cout << ch;
    }
  }
  else cout << "ERROR: Cannot open file: payroll.dat" << endl;
  tfile.close( );
}
```

运行结果：

San Zhang请按任意键继续...

35

## 8.2 输入流

### 6. peek函数

预览将要读入的下一个字符，但不改变输入流。

```
#include <iostream>
using namespace std;
int main( )
{ char c[10], c2;
  c2 = cin.peek( ); // c2获得输入流的第一个字符的值
  cin.getline( &c[0], 9 ); // 读入输入流的字符串
  cout << c2 << " " << c << endl;
}
```

运行时输入：abcdef 输出结果： a abcdef

36

## 8.2 输入流

### 7. putback函数

将一个字符放回到输入流中。但放回去的字符数不能超过读入的字符数。

```
#include <iostream>
using namespace std;
int main()
{ char c[10], c2, c3;
  c2 = cin.get( ); // 读入 a, 输入流为: bcdef\n
  c3 = cin.get( ); // 读入 b, 输入流为: cdef\n
  cin.putback( c2 ); // 将a放回到输入流中, acdef\n
  cin.getline( &c[0], 9 ); // 读入输入流中字符串
  cout << c << endl;
}
运行时输入: abcdef  输出结果: acdef
```

37

```
#include <iostream>
using namespace std;
int main()
{ char c[10], c2, c3;
  c2 = cin.get( );
  c3 = cin.get( );
  cin.putback( 'g' );
  cin.putback( 'h' );
  cin.getline( &c[0], 9 );
  cout << c << endl;
}
```

运行时输入: abcdef  
输出结果: hgcdfe

```
#include <iostream>
using namespace std;
int main()
{ char c[10], c2, c3;
  c2 = cin.get( );
  c3 = cin.get( );
  cin.putback( 'g' );
  cin.putback( 'h' );
  cin.putback( 'k' );
  cin.getline( &c[0], 9 );
  cout << c << endl;
}
```

运行时输入: abcdef  
输出结果:

放回去的字符个数超过从缓冲区读入的字符个数, 结果.....输入缓冲区被毁了

38

## 8.2 输入流

### 8. unget函数

将最近读入的一个字符放回到输入流中。

```
#include <iostream>
using namespace std;
int main()
{ char c[10], c2;;
  c2 = cin.get( ); // 读入 a, 输入流为: bcdef\n
  cin.unget( ); // 将刚读入的字符回到输入流中 abcdef\n
  cin.getline( &c[0], 9 ); // 读入输入流中字符串
  cout << c << endl;
}
运行时输入: abcdef  输出结果: abcdef
```

39

## 8.3 输入输出流

fstream为输入输出流, 它有两个子类:

ifstream(input file stream)  
ofstream(output file stream)

其中 ifstream 默认以输入方式打开文件

ofstream默认以输出方式打开文件。

例如:

ifstream file2("c:\\pdos.def");//以输入方式打开文件

ofstream file3("c:\\x.123");//以输出方式打开文件

所以, 在实际应用中, 根据需要的不同, 选择不同的类来定义:

如果想以输入方式打开, 就用ifstream来定义;

如果想以输出方式打开, 就用ofstream来定义;

如果想以输入/输出方式来打开, 就用fstream来定义。

40

有程序:

```
#include<iostream>
#include<fstream>
using namespace std;
int main( )
{ int a;
  fstream of("data.txt", ios_base::in | ios_base::out);
  of << 111 << " " <<< 222;
  of.seekp(0);
  of >> a;
  cout << a << endl;
}
```

运行结果为: 111

把刚写入文件中的数据111又读入到a中。这里of即是输出对象又是输入对象, 文件打开方式是可读可写的文本方式。

但前提是, data.txt文件在打开时必须先存在, 否则无法保证可读。

除非用: fstream of("data.txt",ios\_base::out | ios\_base::in | ios\_base::trunc);

41

## 第10次作业:

见网络学堂第10次作业

2题必做, 1题选做

42

1. 用C++流操作编程实现从一个文本文件中读入若干个整数，在标准输出设备上按每行10个每个数场宽为8并右对齐输出。
2. 用C++流操作编程实现从一个文本文件中读入若干个整数，用二进制方式输出到文件中，并再从二进制文件中第5个整数开始读入，每次读入2个整数，然后跳过2个整数，循环往复，直至读到文件尾。把读入的数据，在标准输出设备上按每行10个每个数场宽为8并左对齐输出。
- 3\*. 用C++的流操作编写程序从文本文件中读入若干个字符串（每个串长度不超过80个字符），将字符串按字典序（从小到大）排序，结果输出到另一个正文文件中。希望此程序能处理任意多个字符串。

43